

EduTrack2 Project Notes

- EduTrack2 — Complete Project Notes
 - 1. Project Setup
 - 2. Database Setup (MySQL)
 - 3. Folder Structure Overview
 - 4. Job of Each File — `model/` folder
 - 5. Job of Each File — `repository/` folder
 - 6. Job of Each File — `service/` folder
 - 7. Job of Each File — `controller/` folder
 - 8. Job of `config/SecurityConfig.java`
 - 9. Job of the HTML Templates (`templates/` folder)
 - 10. The Complete Data Flow (Student Taking an Exam)
 - 11. Key Concepts Recap
 - 12. Known Limitations / Possible Future Improvements

EduTrack2 — Complete Project Notes

An Online Examination System built with Spring Boot, Thymeleaf, and MySQL. This document explains every file created, what it does, and how the whole system fits together — from the ground up.

1. Project Setup

Tool used: Spring Initializr (start.spring.io)

Setting	Value
Group	com.edutrack2
Artifact	edutrack2
Package	com.edutrack2.edutrack2
Java	17
Build tool	Maven
Packaging	Jar

Dependencies added: - **Spring Web** — lets the app handle HTTP requests and serve web pages (built on Apache Tomcat) - **Spring Data JPA** — connects Java objects to database tables without writing raw SQL - **MySQL Driver** — the actual library that lets Java talk to a MySQL database - **Thymeleaf** — the templating engine that turns HTML files into dynamic pages - **Spring Security** — provides password encryption tools (BCrypt) - **Lombok** — auto-generates getters/setters/constructors so you don't type them by hand - **Validation** — allows form field validation rules (not heavily used yet, but available)

`pom.xml` — Maven's configuration file. Lists all the dependencies above so Maven knows what libraries to download and include in the build. You never edited this by hand; Spring Initializr generated it.

2. Database Setup (MySQL)

- Created a database called `edutrack2` using `CREATE DATABASE edutrack2;`
- Ran into a password issue: `root` user had a saved password in the GUI tool that we didn't know
- **Fix:** created a dedicated MySQL user just for this app:

```
CREATE USER 'edutrack2user'@'localhost' IDENTIFIED BY 'password123';
GRANT ALL PRIVILEGES ON edutrack2.* TO 'edutrack2user'@'localhost';
FLUSH PRIVILEGES;
```

- This user has permission to read/write only the `edutrack2` database — safer than using `root` everywhere.

application.properties — the master configuration file. Tells Spring Boot: - Where the MySQL database lives (`spring.datasource.url`)
- What username/password to connect with - `spring.jpa.hibernate.ddl-auto=update` → Hibernate automatically creates/updates database tables to match your Java classes — no manual `CREATE TABLE` SQL needed - `spring.jpa.show-sql=true` → prints every SQL query Hibernate runs, in the terminal, so you can see what's happening - `server.port=8080` → the app runs at `localhost:8080`

3. Folder Structure Overview

```
src/main/java/com/edutrack2/edutrack2/
├── Edutrack2Application.java    ← the entry point (main method)
├── config/
│   └── SecurityConfig.java
├── model/                      ← Java classes = database tables
│   ├── User.java
│   ├── Admin.java
│   ├── Test.java
│   ├── Question.java
│   ├── Notification.java
│   └── Activity.java
├── repository/                 ← auto-generated database queries
│   ├── UserRepository.java
│   ├── AdminRepository.java
│   ├── TestRepository.java
│   ├── QuestionRepository.java
│   ├── NotificationRepository.java
│   └── ActivityRepository.java
├── service/                   ← business logic
│   ├── UserService.java
│   ├── AdminService.java
│   ├── TestService.java
│   ├── NotificationService.java
│   └── ActivityService.java
└── controller/                ← handles web requests
    ├── AuthController.java
    ├── DashboardController.java
    ├── AdminAuthController.java
    ├── AdminTestController.java
    ├── AdminNotificationController.java
    ├── AdminActivityController.java
    └── ExamController.java

src/main/resources/
├── application.properties
├── static/css/style.css
└── templates/                 ← HTML pages (Thymeleaf)
    ├── login.html
    ├── register.html
    ├── dashboard.html
    ├── test-series.html
    ├── activities.html
    ├── notifications.html
    ├── performance.html
    ├── exam.html
    ├── exam-result.html
    ├── admin-login.html
    ├── admin-dashboard.html
    ├── admin-tests.html
    ├── admin-test-new.html
    ├── admin-add-question.html
    ├── admin-notifications.html
    └── admin-activities.html
```

This follows the **MVC pattern** (Model-View-Controller): - **Model** = your data structure (the `model/` folder — Java classes representing database tables) - **View** = what the user sees (the `templates/` folder — HTML pages) - **Controller** = the middleman that receives a request, talks to services/repositories, and decides which view to show (the `controller/` folder)

4. Job of Each File — `model/` folder

These classes represent **database tables**. Each field becomes a column. `@Entity` tells Spring “this class = a table.”

`User.java`

Represents a student account. Fields: `id`, `username`, `password`, `email`. The `username` and `email` are marked `unique = true` so no two students can register with the same one.

`Admin.java`

Represents an admin account — completely separate table from `User`. Fields: `id`, `username`, `password`. Kept separate so admins and students never share login logic or get mixed up.

`Test.java`

Represents one exam/test (e.g., “Java Basics Quiz”). Fields: `id`, `testName`, and a `List<Question> questions`. The `@OneToMany` annotation means **one Test has many Questions** — Hibernate manages this relationship automatically.

`Question.java`

Represents a single MCQ question. Fields: `questionText`, `optionA/B/C/D`, `correctOption` (stores “A”/“B”/“C”/“D”), and a `@ManyToOne` link back to its parent `Test` (via a `test_id` foreign key column).

`Notification.java`

Represents an announcement posted by admin. Fields: `title`, `message`, `createdAt` (auto-set to the current time when created).

`Activity.java`

Represents an assignment/task posted by admin. Fields: `title`, `description`, `createdAt`.

Why `@Data` (Lombok) on every class? It auto-generates `getX()`, `setX()`, `toString()`, `equals()`, and `hashCode()` at compile time — saves you from typing dozens of boilerplate methods by hand.

5. Job of Each File — `repository/` folder

These are **interfaces** (not classes) that extend `JpaRepository<EntityType, IdType>`. You don’t write any implementation — Spring generates one automatically at runtime.

`UserRepository.java`

Gives you `.save()`, `.findAll()`, `.findById()` for free. Also defines custom lookups: - `findByUsername(String)` — used during login to find a user by their username - `existsByUsername()` / `existsByEmail()` — used during registration to check for duplicates

How does `findByUsername` work with no code inside it? Spring reads the method name itself and builds the SQL query automatically. This is called a **derived query method**.

`AdminRepository.java`

Same idea, but for the `Admin` table — `findByUsername`, `existsByUsername`.

`TestRepository.java`

Basic repository for `Test` — no custom methods needed since `.findAll()` and `.findById()` cover everything we use.

`QuestionRepository.java`

Basic repository for `Question` — used internally when saving new questions.

`NotificationRepository.java`

Adds `findAllOrderByCreatedAtDesc()` — fetches all notifications, newest first.

ActivityRepository.java

Same pattern as notifications — `findAllByOrderByCreatedAtDesc()` .

6. Job of Each File — `service/` folder

Services hold the **business logic** — the actual decision-making, sitting between controllers (which handle web requests) and repositories (which touch the database).

UserService.java

- `registerUser()` — checks if username/email already exist; if not, encrypts the password with `BCryptPasswordEncoder` and saves the new user
- `validateLogin()` — looks up the user by username, then checks if the typed password matches the encrypted one using `.matches()`

AdminService.java

- `createDefaultAdminIfMissing()` — creates a default admin account (`admin` / `admin123`) the first time `/admin/setup` is visited, only if it doesn't already exist
- `validateLogin()` — same pattern as `UserService` , but for the `Admin` table

TestService.java

- `createTest()` — saves a new `Test` with just a name
- `getAllTests()` / `getTestById()` — fetch tests for display
- `addQuestion()` — builds a `Question` object from form data, links it to its parent `Test` , and saves it
- `calculateScore()` — loops through every question in a test, compares the student's submitted answer to the correct one, and counts how many are right

NotificationService.java

- `addNotification()` — saves a new notification
- `getAll()` — fetches all notifications, newest first

ActivityService.java

- `addActivity()` — saves a new activity
 - `getAll()` — fetches all activities, newest first
-

7. Job of Each File — `controller/` folder

Controllers receive **HTTP requests** (when someone visits a URL or submits a form) and decide what to do — call a service, then return the name of an HTML template to display.

AuthController.java (student login/register)

- `/` → redirects to `/login`
- `/login` (GET) → shows the login page
- `/login` (POST) → validates credentials, stores `"loggedInUser"` in the session if correct, redirects to dashboard
- `/register` (GET) → shows the registration form
- `/register` (POST) → creates the account via `UserService`
- `/logout` → clears the session, sends back to login

DashboardController.java (student-facing pages)

Every method here checks `session.getAttribute("loggedInUser")` first — if not logged in, redirects to `/login` . - `/dashboard` → shows the 4-option dashboard - `/test-series` → fetches all tests via `TestService` and displays them - `/activities` → fetches all activities via `ActivityService` - `/notifications` → fetches all notifications via `NotificationService` - `/performance` → placeholder page (no score history stored yet)

AdminAuthController.java (admin login)

- `/admin/setup` → one-time URL that creates the default admin account
- `/admin/login` (GET/POST) → separate login flow from students, stores `"loggedInAdmin"` in session (a completely different session key than students use)
- `/admin/logout` → clears the admin session

`AdminTestController.java` (admin builds tests & questions)

Every method checks `session.getAttribute("loggedInAdmin")` first. - `/admin/dashboard` → shows the 3-option admin panel - `/admin/tests` → lists all created tests - `/admin/tests/new` (GET/POST) → form to create a new test, then redirects into the question-adding screen - `/admin/tests/{id}/questions` (GET/POST) → shows the “add a question” form and saves each submitted question, one at a time

`AdminNotificationController.java`

- `/admin/notifications` (GET) → shows the form + list of existing notifications
- `/admin/notifications` (POST) → saves a new notification

`AdminActivityController.java`

Same pattern as notifications, but for activities.

`ExamController.java` (student takes the exam)

Checks `session.getAttribute("loggedInUser")` first. - `/exam/{id}` (GET) → loads a specific test and displays all its questions as a form - `/exam/{id}/submit` (POST) → reads every submitted answer from the form, compares against correct answers via `TestService.calculateScore()`, and shows the result page

8. Job of `config/SecurityConfig.java`

- Defines a `PasswordEncoder` bean (`BCryptPasswordEncoder`) — used everywhere passwords are saved or checked
- Configures Spring Security to **allow every request** (`permitAll()`) and **disables CSRF** — because we built our own custom login system using `HttpSession` instead of using Spring Security’s built-in login form
- This is a simplification appropriate for a learning project; a production app would re-enable CSRF protection and harden this further

9. Job of the HTML Templates (`templates/` folder)

All pages use **Thymeleaf** syntax (`th:text` , `th:each` , `th:if` , `th:action`) to inject dynamic data from the controller into plain HTML.

File	Purpose
<code>login.html</code>	Student login form
<code>register.html</code>	Student registration form
<code>dashboard.html</code>	Shows the 4 option cards (Test Series, Activities, Notifications, Performance)
<code>test-series.html</code>	Lists all available tests; clicking one goes to <code>/exam/{id}</code>
<code>activities.html</code>	Lists all activities posted by admin
<code>notifications.html</code>	Lists all notifications posted by admin
<code>performance.html</code>	Placeholder — future home for score history
<code>exam.html</code>	Renders every question in a test as radio-button options, wrapped in one form
<code>exam-result.html</code>	Shows the final score after submission
<code>admin-login.html</code>	Admin login form
<code>admin-dashboard.html</code>	Shows the 3 option cards (Add Exam, Add Notifications, Add Activities)
<code>admin-tests.html</code>	Lists all tests admin has created, with an “Edit” link to add more questions
<code>admin-test-new.html</code>	Form to name a new test

admin-add-question.html	Form to add one question + 4 options + correct answer at a time
admin-notifications.html	Form + list for posting notifications
admin-activities.html	Form + list for posting activities

`static/css/style.css` — one shared stylesheet used by every page (linked via `<link rel="stylesheet" href="/css/style.css">`), giving the whole app a consistent visual theme instead of scattered inline styles.

10. The Complete Data Flow (Student Taking an Exam)

1. Admin logs in → creates a `Test` → adds several `Question`s to it (all saved to MySQL)
2. Student logs in → clicks **Test Series** → `DashboardController.testSeries()` fetches all tests and displays them
3. Student clicks a test name → `ExamController.takeExam()` loads that specific test with all its questions → renders `exam.html`
4. Student selects an answer for each question, clicks **Submit Exam**
5. `ExamController.submitExam()` reads every submitted answer from the form, calls `TestService.calculateScore()` to compare against the correct answers stored in the database
6. Result (score out of total) is displayed on `exam-result.html`

11. Key Concepts Recap

- **Spring Boot** auto-configures a working web app with minimal manual setup
- **JPA/Hibernate** maps Java classes to database tables, so you rarely write raw SQL
- **Repositories** give free CRUD operations, plus auto-generated queries from method names
- **Services** hold business logic — the “brains” between controllers and the database
- **Controllers** are the entry point for every URL — they decide what data to fetch and what page to show
- **Thymeleaf** turns plain HTML into dynamic pages using simple `th:` attributes
- **Sessions** (`HttpSession`) are how the server “remembers” who’s logged in across different page requests
- **BCrypt** ensures passwords are never stored as plain, readable text

12. Known Limitations / Possible Future Improvements

- No exam attempt history — scores aren’t saved per student after viewing the result once
- No timer on exams
- Default admin credentials are hardcoded (`admin` / `admin123`) rather than set securely
- CSRF protection is disabled for simplicity
- No pagination — all tests/notifications/activities load at once, which won’t scale well with large amounts of data
- Only single-correct-answer MCQs are supported (no checkbox/multi-select questions)